

A Technical Study of the Development of RUNSHEET — EMS Field

Reference

Author: Dominique Ball

Corresponding Author Email: Domhasaball@gmail.com

Article Type: Viewpoint / Tutorial

1. Introduction

RUNSHEET is a web-based educational platform designed for EMT and paramedic students.

The application combines several different learning and decision-support features into a single interface, including patient assessment, symptom-to-impression matching, ECG rhythm recognition, medication reference information, and an AI-powered copilot.

The website describes itself as a **study tool rather than a clinical protocol**. It emphasizes that its recommendations are based on general EMS reference material and that users should follow their local protocols, treatment guidelines, and medical direction.

This study examines the publicly observable functionality of RUNSHEET and proposes how the system could have been architected and implemented. Because the application's private source code, database, deployment configuration, and development history are not publicly available, implementation details that cannot be directly observed are identified as likely architectural approaches rather than confirmed facts.

2. Purpose of the Application

The primary objective of RUNSHEET appears to be providing EMS students with an interactive environment for practicing patient assessment and reviewing common emergency-care concepts.

Instead of presenting information as a traditional static reference manual, the application allows the user to construct a patient scenario.

The main workflow can be summarized as:

Patient information → Clinical findings → Vitals/ECG → Assessment → Ranked impressions → Educational treatment reference

The homepage describes the central assessment feature as allowing users to enter vital signs and select signs, symptoms, and risk factors. The system then ranks conditions that best match the selected findings and displays associated field-treatment information, oxygen targets, and medication doses.

This makes the application more interactive than a conventional medical reference website.

3. Major Components

The public interface exposes several major components.

3.1 Patient Assessment

The Patient section allows users to enter clinical findings and vital signs.

The application indicates that vital signs are optional and that abnormal values can automatically be added as findings. This suggests that the application contains a rule-based preprocessing layer that interprets numerical inputs before running the assessment.

For example, conceptually:

User enters vital signs



Validate input



Identify abnormal values



Convert abnormalities into findings



Combine with selected symptoms



Calculate condition matches



Rank possible impressions

This is an important design decision because it allows the user to provide information in numerical form while the assessment engine works with higher-level clinical findings.

4. Symptom-to-Impression Engine

One of the most important components appears to be the **Symptom** → **Impression** engine.

The user selects signs, symptoms, and risk factors. The application then determines which conditions are most compatible with those findings.

A likely implementation would represent each condition as a collection of associated findings.

For example, a simplified internal structure could look conceptually like:

Condition A

finding_1

finding_2

finding_3

Condition B

finding_2

finding_4

finding_5

When a user selects findings, the application can compare the patient's findings against the condition definitions.

A simple scoring approach could be:

score =

number of matching findings
/ number of relevant findings

The conditions could then be sorted from the highest score to the lowest score.

This would allow RUNSHEET to produce a ranked differential-style educational result without requiring a complex machine-learning model.

The website describes the results as conditions that "best match" the information entered, which is consistent with a matching or scoring architecture.

5. Full Scenario Builder

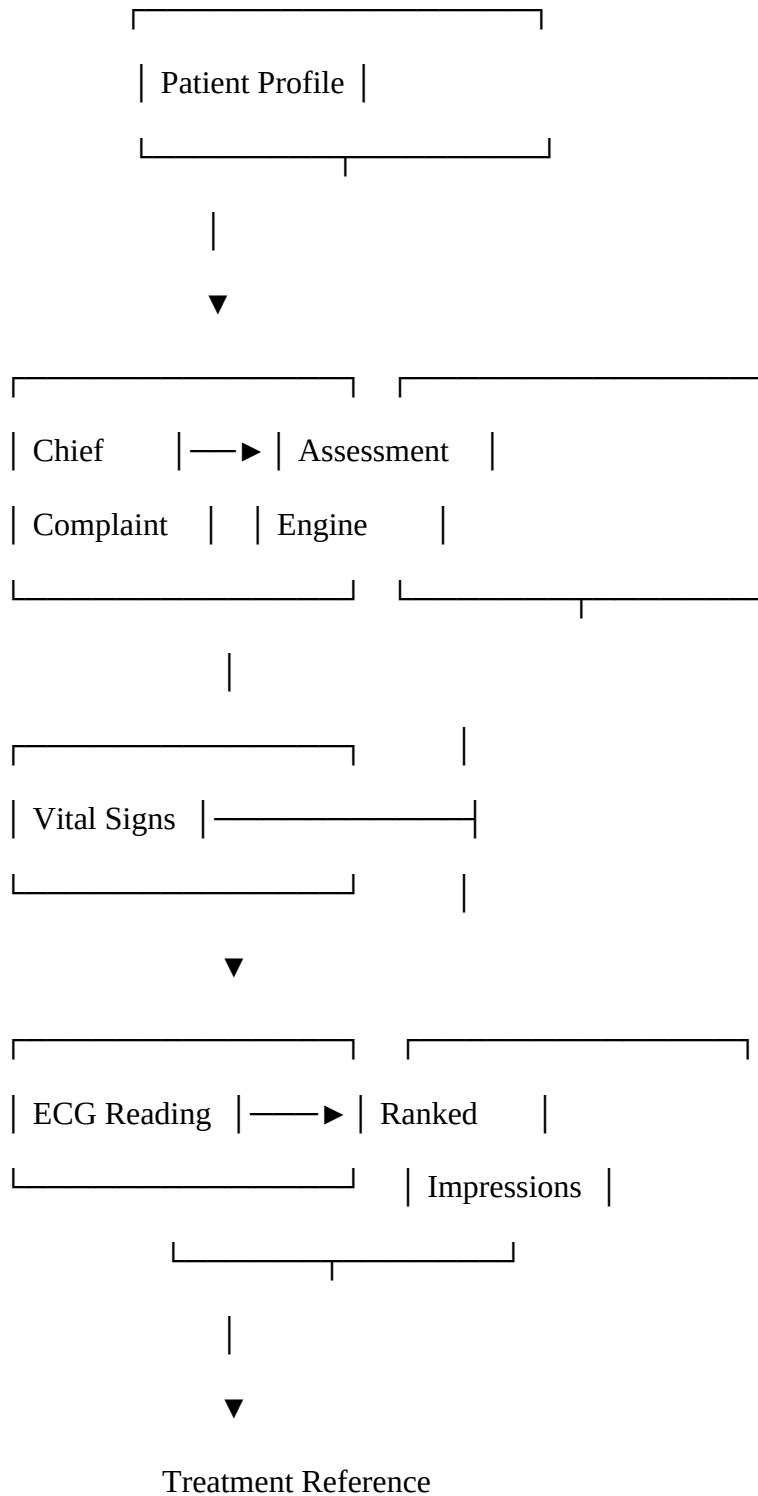
RUNSHEET also provides a more comprehensive **Full Scenario** workflow.

The public interface states that users can enter:

- Demographics
- Chief complaint findings
- Vital signs
- ECG information

The system then combines the information and produces a ranked impression with field-treatment information.

A conceptual architecture would therefore be:



This architecture allows information from multiple parts of a patient presentation to be considered together.

6. Vital Sign Processing

The application supports several common vital-sign measurements.

The public interface exposes heart rate and SpO₂ information and indicates that abnormal vital signs can automatically become assessment findings.

A likely implementation uses predefined ranges or thresholds.

For example, conceptually:

```
if heart_rate < lower_limit:
```

```
    findings.append("bradycardia")
```

```
if heart_rate > upper_limit:
```

```
    findings.append("tachycardia")
```

```
if spo2 < oxygen_threshold:
```

```
    findings.append("low oxygen saturation")
```

The resulting findings can then be passed to the same assessment engine used for manually selected symptoms.

This design avoids having two separate assessment systems.

7. ECG Rhythm Recognition

Another major feature is the cardiac rhythm reference.

The website provides a **Cardiac Monitor & Rhythm Reference** section containing a live monitor-style interface and a rhythm library. Users can select a rhythm and view a corresponding monitor trace along with associated vital signs.

The site specifically explains that the rhythm strips are stylized for pattern recognition and are not calibrated clinical tracings.

This suggests that the ECG component was designed primarily as an educational visualization rather than as an actual ECG analysis system.

A likely implementation would use predefined rhythm objects containing information such as:

Rhythm

|— Name

|— Heart rate

|— SpO₂

|— ECG pattern

|— Clinical description

|— Treatment reference

The front end can then render the selected rhythm dynamically.

8. ECG Visualization

The live monitor is likely implemented using browser-based graphics rather than displaying a collection of static images.

A web application could generate an ECG-like waveform using:

- HTML Canvas
- SVG
- CSS animation
- JavaScript rendering

A simplified conceptual process would be:

Selected rhythm



Rhythm configuration



Waveform parameters



JavaScript rendering



Animated monitor display

Different rhythm configurations could produce different waveform shapes.

For an educational application, this approach has the advantage of allowing the same visualization component to display many rhythms without requiring a separate image for every scenario.

9. Medication Reference System

RUNSHEET also contains a medication dosage guide.

The site describes this section as a reference containing common adult emergency doses and provides search/filter functionality.

A likely implementation would store medication information in a structured data source.

For example:

Medication

└─ Name

└─ Category

└─ Adult dose

└─ Route

└─ Indications

└─ Contraindications
└─ Notes

The user interface can then filter the dataset when a medication name or category is entered.

The site explicitly warns that the displayed doses are common adult reference ranges and that pediatric dosing, contraindications, routes, and maximum doses can vary.

This is particularly important because medication information is safety-critical.

10. AI Copilot

RUNSHEET includes a **Paramedic AI Copilot**.

The public page indicates that the copilot provides decision-support answers using the same knowledge base referenced throughout the site.

Interestingly, the publicly visible page embeds the AI component separately rather than displaying the entire AI interface as native page content. The page contains an iframe pointing to a Streamlit application.

This suggests an architecture similar to:

RUNSHEET Website

|
| iframe



AI Copilot Application



Knowledge Base / AI System

Using an iframe allows the main website and AI application to be developed and deployed separately.

The visible page also identifies the AI component as a demonstration copilot and emphasizes that it is not a replacement for training, protocols, or medical direction.

11. Front-End Architecture

Based on the publicly observable interface, RUNSHEET appears to use a modern single-page web application approach.

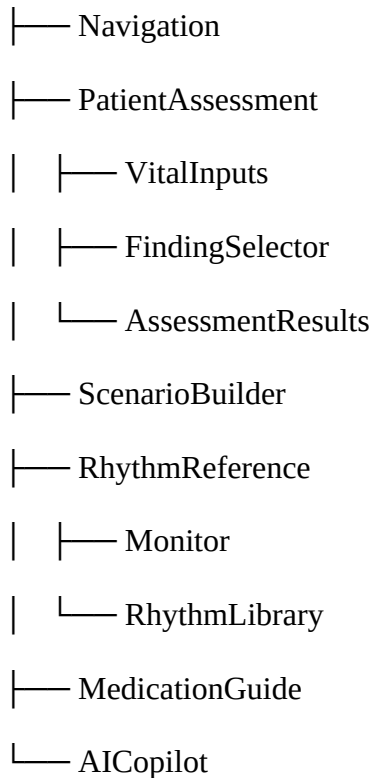
The interface contains multiple interactive sections:

- Patient assessment
- Full scenario
- Rhythm recognition
- Medication search
- AI Copilot

The page dynamically updates controls and results rather than requiring a separate page for every individual clinical condition.

A likely front-end architecture could therefore consist of reusable components such as:

App



This component-based architecture would make it easier to maintain and extend the application.

12. Data Architecture

The application requires structured clinical knowledge rather than relying entirely on free-form text.

The knowledge base likely contains relationships between:

Symptoms



Conditions



Treatment References



Medications

For example:

Finding



├── Condition A

| ├── Treatment

| └── Medication



└── Condition B

 ├── Treatment

 └── Medication

This structure allows the same underlying information to be reused by several features.

For example, the assessment engine can use condition data while the AI Copilot can use the same knowledge base to answer educational questions.

The website explicitly states that the AI Copilot uses the same knowledge base referenced throughout the site.

13. Likely Technology Stack

The exact private technology stack cannot be established from the public interface alone.

However, some components can be identified directly.

The AI Copilot is embedded from a **Streamlit application**, indicating that at least that component uses Streamlit.

A plausible overall architecture is therefore:

Layer	Likely Technology/Approach
Main website	Modern HTML/CSS/JavaScript web application
Interactive UI	JavaScript-based components
Clinical knowledge	Structured JSON/database
Assessment engine	Rule-based scoring/matching
ECG visualization	Browser-based graphics/animation
Medication search	Structured searchable dataset

AI Copilot	Python/Streamlit application
AI knowledge	Shared/reference knowledge base
Hosting	Web hosting + separate AI application hosting

The items described as "likely" cannot be confirmed without access to the project's source code or deployment configuration.

14. Why a Rule-Based System Makes Sense

A particularly important architectural observation is that the core patient assessment does not necessarily need machine learning.

For an educational EMS application, a transparent rule-based system has several advantages.

A rule can be inspected:

If finding X is present,
increase the score for condition Y.

This is much easier to explain than an opaque prediction.

It also allows the developer to control exactly which findings are associated with each educational impression.

A simple scoring system could therefore be sufficient:

```
score = 0
```

```
for finding in patient_findings:
```

```
    if finding in condition_findings:
```

```
        score += condition_findings[finding]
```

The conditions can then be ranked according to their scores.

This approach also makes it easier to update educational content when reference material changes.

15. User Experience Design

The application appears to have been designed around the workflow of an EMS student.

Rather than presenting hundreds of pages of reference information, the application organizes information around tasks:

Assess → Interpret → Review → Learn

The user can start with symptoms, build an entire patient scenario, practice rhythm recognition, search medications, or ask the AI Copilot a question.

This modular design makes the website useful as both a reference tool and a study environment.

The homepage also repeatedly labels the system as a study tool rather than a protocol, which establishes an important boundary between educational software and clinical decision-making.

16. Safety Considerations

Because the application deals with medical information, safety is a central part of its design.

The website repeatedly warns users that the information should not replace:

- Training
- Certification standards
- Local protocols
- Medical direction
- Professional clinical judgment

The medication section also specifically warns users to confirm doses against their agency's protocols before administering medication.

This type of disclaimer is particularly important when an application provides medication doses or treatment recommendations.

The application should therefore be understood as an educational reference rather than a medical device or autonomous clinical decision-making system.

17. Proposed Development Process

A project similar to RUNSHEET could be developed in the following stages.

Stage 1 — Requirements

Identify the target audience and define the major features:

- Patient assessment
- Scenario builder
- ECG library
- Medication reference
- AI Copilot

Stage 2 — Knowledge Base

Create structured datasets for:

- Clinical findings
- Conditions
- Treatments
- Medications
- ECG rhythms

Stage 3 — Assessment Engine

Implement the symptom-to-impression matching system and scoring logic.

Stage 4 — Front-End

Build the interactive user interface and connect the controls to the assessment engine.

Stage 5 — ECG Visualization

Create reusable waveform/monitor components for the rhythm library.

Stage 6 — Medication Search

Implement searchable and filterable medication information.

Stage 7 — AI Integration

Develop the AI Copilot separately and connect it to the site's reference knowledge.

The publicly visible Streamlit iframe suggests that the AI component can be deployed independently from the main website.

Stage 8 — Testing

Test:

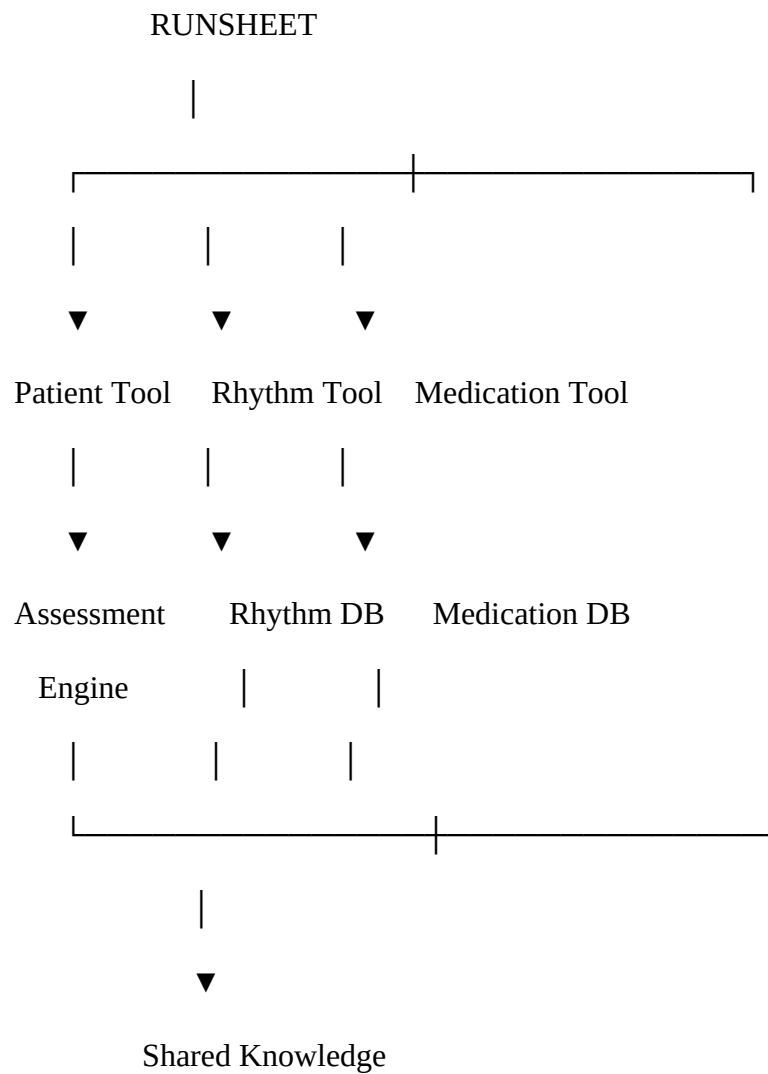
- Input validation
- Assessment results
- Ranking behavior
- ECG rendering
- Medication searches
- AI responses
- Mobile responsiveness
- Safety messaging

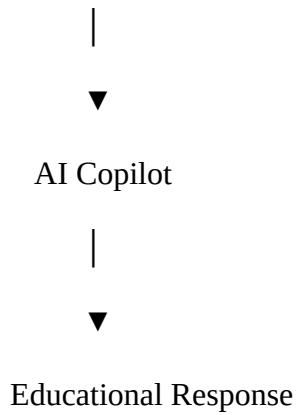
Stage 9 — Deployment

Deploy the main website and AI application and connect the components through the web interface.

18. Example System Architecture

The overall system can be represented as:





This architecture separates the major responsibilities while allowing them to share a common clinical knowledge base.

19. Strengths of the Design

Several aspects of the observed design are particularly effective.

Interactive learning

Users actively construct scenarios instead of simply reading information.

Modular architecture

Assessment, ECG, medications, and AI are separate functional areas.

Reusable knowledge

The same clinical information can support multiple features.

Visual learning

The ECG monitor provides a visual way to practice rhythm recognition.

Clear safety boundaries

The application repeatedly explains that it is a study tool and not a replacement for protocols or medical direction.

AI integration

The AI Copilot provides a conversational interface for exploring the same educational knowledge base.

20. Limitations of This Study

This study is based on publicly observable behavior and content from RUNSHEET.

The private source code, database schema, version-control history, backend implementation, AI model configuration, hosting infrastructure, and developer documentation were not available for inspection.

Consequently, statements about specific frameworks or algorithms beyond what can be directly observed should be considered proposed or likely implementations rather than confirmed facts.

The publicly visible site does, however, provide enough information to identify the major functional architecture and to develop a reasonable technical model of how the application could have been constructed.

21. Conclusion

RUNSHEET demonstrates how an EMS educational application can combine structured clinical knowledge, interactive assessment, visualization, searchable references, and artificial intelligence into one web-based platform.

Its core architecture can be understood as several interconnected components: a patient assessment engine, a scenario builder, an ECG rhythm visualization system, a medication reference database, and an independently embedded AI Copilot.

The assessment functionality appears well suited to a rule-based matching architecture, while the ECG system can use predefined rhythm data and browser-based visualization. The medication guide can operate from a structured searchable dataset, and the AI Copilot can be deployed as a separate application connected to the site's educational knowledge.

The result is a system that transforms a conventional EMS reference guide into an interactive learning environment.

Most importantly, RUNSHEET illustrates that an effective medical education application does not necessarily require a complex machine-learning model at its core. A carefully structured

knowledge base, transparent rules, intuitive user interface, and appropriately bounded AI system can together provide a powerful educational experience.

Source examined: RUNSHEET — EMS Field Reference, publicly available at runsheets.com. The site identifies itself as an EMS study tool and states that its information should be verified against local protocols and medical direction.